

Aula 04 — Operadores e Expressões

Sistema de Registro de Vendas da Feira Flutuante do Rio Negro

Habilidade e Objetos de Conhecimento

HABILIDADE

Compreender a representação gráfica de algoritmos por meio de diagramas de blocos, bem como o processo de tradução de um algoritmo em instruções executáveis pelo computador, reconhecendo a estrutura universal de entrada, processamento e saída de dados.

OBJETO

Diagrama de Blocos e Quadros; Compiladores, Interpretadores e Tradutores; Etapas de um Programa: Entrada de Dados, Processamento e Saída de Dados.

Desafio da Fase 1: "Cálculo de Vendas"

- 1 A partir de hoje, nosso sistema precisa calcular o valor de uma venda na feira
- 2 Isso envolve multiplicar preço por quantidade, somar itens diferentes e calcular troco
- 3 Para isso, precisamos dominar o operador de atribuição e os operadores aritméticos
- 4 Vamos construir, passo a passo, as primeiras expressões desse módulo

Ordem de precedência dos operadores

- 1 Multiplicação (*) e divisão (/) são calculadas antes de adição (+) e subtração (-)
- 2 Parênteses () sempre têm prioridade máxima e podem forçar uma ordem diferente
- 3 Operações de mesma prioridade são calculadas da esquerda para a direita
- 4 Ignorar essa ordem é uma das causas mais comuns de erro em algoritmos

Atribuição x Igualdade — não confunda!

Atribuição (←)

- Guarda um valor na variável
- Usada para calcular e armazenar
- Ex.: total ← 45.00

Igualdade relacional (=)

- Compara dois valores
- Retorna VERDADEIRO ou FALSO
- Ex.: total = 45.00

1º

Diagrama de Blocos e Quadros

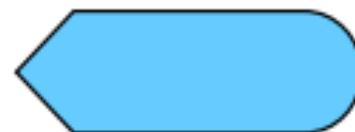
Representando algoritmos graficamente

Fluxograma - Simbologia

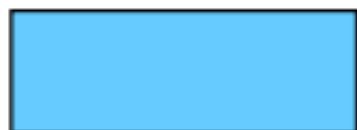
- Cada instrução ou ação a ser executada deve ser representada por meio de um símbolo gráfico.



Terminal: representa o início e o final do fluxograma.



Vídeo: representa a saída de informações por meio do monitor de vídeo.



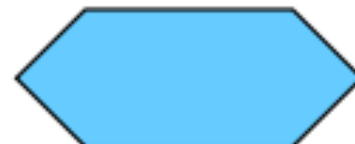
Processamento: representa a execução de operações ou ações.



Decisão: representa uma ação lógica que resultará na escolha de uma das seqüências de instruções.

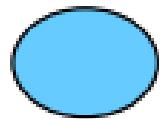


Teclado: representa a entrada de dados para as variáveis por meio do teclado.



Preparação: representa uma ação de preparação para o processamento.

Fluxograma - Simbologia



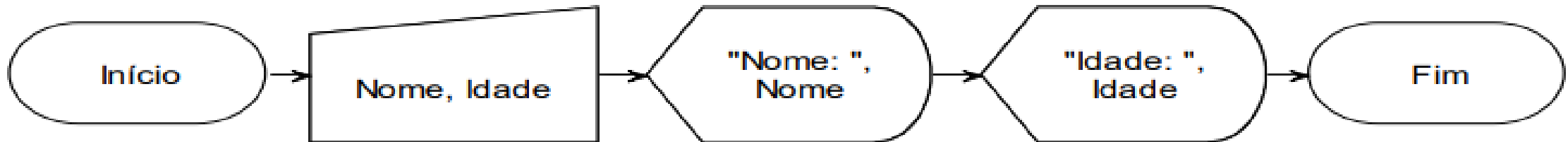
Conector: utilizado para interligar partes do fluxograma ou para desviar o fluxo corrente para um determinado trecho do fluxograma.



Conector de Páginas: utilizado para interligar partes do fluxograma em páginas distintas.



Seta de orientação do fluxo.



O que é um diagrama de blocos?

- 1 É uma representação gráfica simplificada das grandes etapas de um algoritmo ou sistema
- 2 Usa quadros (retângulos) conectados por setas, indicando a sequência de execução
- 3 É mais simples que um fluxograma — foca na visão geral, não em cada detalhe da lógica
- 4 Muito usado para explicar rapidamente "o que o sistema faz", antes de detalhar "como"

Diagrama de Blocos x Fluxograma

Diagrama de Blocos

- Visão geral, macro
- Poucos blocos, grandes etapas
- Ideal para apresentar o sistema

Fluxograma

- Visão detalhada, micro
- Muitos símbolos (decisão, laço...)
- Ideal para representar a lógica completa

Elementos de um diagrama de blocos

- 1 Quadros (retângulos): representam uma etapa, módulo ou processo do sistema
- 2 Setas: indicam a ordem e o sentido do fluxo entre as etapas
- 3 Rótulos: nomeiam cada quadro de forma clara e objetiva
- 4 Alguns diagramas agrupam quadros em "camadas" (entrada, processamento, saída)

Por que usar um diagrama de blocos antes de programar?

- 1 Ajuda a enxergar o sistema como um todo, antes de se perder em detalhes de código
- 2 Facilita a comunicação com colegas de equipe e com quem não é da área técnica
- 3 Serve como um "mapa" para organizar em que ordem os módulos serão desenvolvidos
- 4 É a primeira ferramenta usada em projetos reais de desenvolvimento de sistemas

Diagrama de blocos x diagrama de quadros

- 1 Os termos "diagrama de blocos" e "diagrama de quadros" costumam ser usados como sinônimos
- 2 Ambos representam etapas do sistema por meio de caixas conectadas
- 3 Alguns materiais diferenciam: "quadros" quando o foco é a estrutura de dados; "blocos" quando o foco é o processo
- 4 O importante é a função: simplificar a visão geral de um sistema complexo

1

Desafio do diagrama

Representem, em um diagrama de blocos, as etapas de uma venda completa na feira: desde o cliente escolher o produto até receber o troco e o comprovante.

Dicas para um bom diagrama de blocos

- 1 Cada quadro deve representar apenas UMA etapa — evite misturar duas ações no mesmo bloco
- 2 Use verbos de ação nos rótulos (ex.: "Calcular total", "Emitir comprovante")
- 3 Mantenha as setas em um único sentido sempre que possível, para facilitar a leitura
- 4 Revise: alguém de fora conseguiria entender o sistema só olhando o diagrama?

RECAPITULANDO

Diagrama de Blocos e Quadros

- ✓ É uma representação gráfica simplificada, com quadros e setas
- ✓ Foca na visão geral das grandes etapas de um sistema, não nos detalhes da lógica
- ✓ É diferente (e mais simples) do que o fluxograma
- ✓ Ajuda a planejar e comunicar um sistema antes de detalhar seu código

2º

Prática de Diagramas e Introdução aos Tradutores

Do papel para o computador

Correção da Atividade de Diagrama

- 1 Cada dupla apresenta seu diagrama de blocos da venda da feira
- 2 Vamos comparar diferentes formas corretas de representar o mesmo processo
- 3 Reforçaremos a diferença entre "etapa geral" (bloco) e "passo detalhado" (fluxograma)
- 4 Guardem o diagrama — ele ajudará a planejar os próximos módulos do sistema

E depois do diagrama, o que acontece?

- 1 Depois de planejar com diagramas e escrever o pseudocódigo, chega a hora de programar de verdade
- 2 Mas o computador não entende português, nem pseudocódigo, nem a maioria das linguagens de programação diretamente
- 3 É preciso "traduzir" o código escrito por nós para a linguagem que o processador realmente entende
- 4 Essa tradução é feita por programas especializados, que veremos agora

O que é linguagem de máquina?

- 1 É a única linguagem que o processador do computador executa diretamente
- 2 É formada por sequências de números binários (0s e 1s)
- 3 Nenhum ser humano programa diretamente em linguagem de máquina no dia a dia
- 4 Por isso, usamos linguagens mais próximas do português ou do inglês, que depois são traduzidas

O que é um tradutor de linguagem?

- 1 É um programa que converte um código escrito em uma linguagem de programação para outra linguagem
- 2 O destino mais comum é a linguagem de máquina, que o processador entende
- 3 Existem diferentes tipos de tradutores, cada um com uma forma própria de trabalhar
- 4 Os três principais tipos são: compilador, interpretador e montador (assembler)

Do código-fonte à execução



Por que isso importa para quem está aprendendo lógica?

- 1 Entender esse processo ajuda a compreender mensagens de erro e o comportamento do Portugol Studio
- 2 Explica por que alguns erros só aparecem "na hora de rodar" e outros "antes mesmo de rodar"
- 3 É um conhecimento fundamental para quem for atuar futuramente com desenvolvimento de sistemas
- 4 Prepara você para entender qualquer linguagem de programação real no futuro

RECAPITULANDO

Introdução aos Tradutores

- ✓ O processador só entende linguagem de máquina (0s e 1s)
- ✓ Um tradutor converte nosso código para essa linguagem
- ✓ O Portugol Studio já faz essa tradução por trás dos panos, ao "Executar"
- ✓ A seguir, vamos detalhar os dois principais tipos de tradutor: compilador e interpretador

3º

Compiladores, Interpretadores e Tradutores

Como o computador entende nosso código

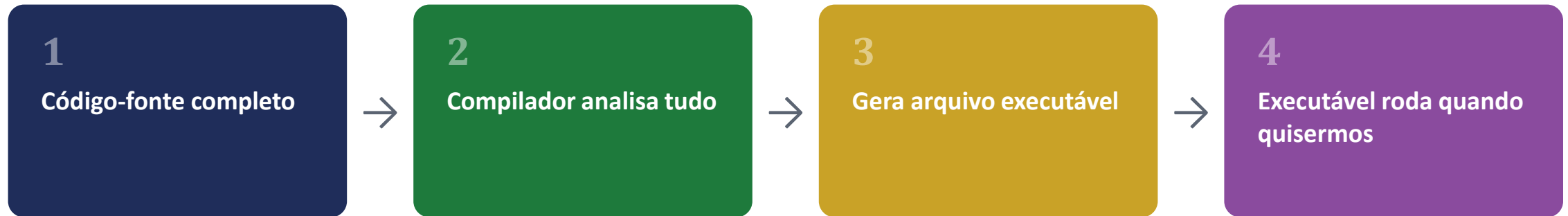
Os três tipos principais de tradutores

- 1 Compilador: traduz todo o código de uma vez, antes da execução
- 2 Interpretador: traduz e executa o código linha por linha, durante a execução
- 3 Montador (Assembler): traduz uma linguagem de baixo nível (Assembly) para linguagem de máquina
- 4 Vamos aprofundar os dois primeiros, mais relevantes para o nosso curso

O que é um compilador?

- 1 É um tradutor que lê todo o código-fonte de uma vez e gera um arquivo executável
- 2 Esse arquivo pode ser executado depois, sem precisar do compilador novamente
- 3 Se houver erro de sintaxe, o compilador aponta o problema ANTES de gerar o executável
- 4 Exemplos de linguagens compiladas: C, C++, Java (parcialmente) e Portugol Studio (em parte)

Como funciona um compilador



O que é um interpretador?

- 1 É um tradutor que lê e executa o código linha por linha, sem gerar um executável separado
- 2 Cada vez que o programa é executado, o interpretador traduz tudo novamente
- 3 Se houver um erro em uma linha, o programa executa normalmente até chegar nela
- 4 Exemplos de linguagens interpretadas: Python, JavaScript e o próprio Portugol (modo de simulação)

Como funciona um interpretador



Compilador x Interpretador

Compilador

- Traduz tudo de uma vez
- Gera um executável separado
- Erros aparecem antes de rodar

Interpretador

- Traduz e executa linha a linha
- Não gera executável separado
- Erros aparecem durante a execução

E o Montador (Assembler)?

- 1 Traduz a linguagem Assembly (muito próxima da linguagem de máquina) para código executável
- 2 É usado em situações muito específicas: sistemas embarcados, drivers, programação de baixo nível
- 3 Não faremos uso dele neste curso, mas é importante saber que ele existe
- 4 Compilador, interpretador e montador são todos exemplos de "tradutores de linguagem"

Onde o Portugol se encaixa?

- 1 O Portugol Studio funciona principalmente como um interpretador educacional
- 2 Ele lê nosso pseudocódigo e o executa passo a passo, mostrando o resultado imediatamente
- 3 Isso facilita identificar exatamente onde um erro ocorreu durante a simulação
- 4 Esse comportamento é parecido com o de linguagens reais interpretadas, como Python

1

Compilado ou interpretado?

O programa gera um arquivo separado, que pode ser executado depois sem precisar do código-fonte original.

2

Compilado ou interpretado?

Ao rodar o programa, um erro na linha 20 só aparece depois que as 19 linhas anteriores já foram executadas normalmente.

4º

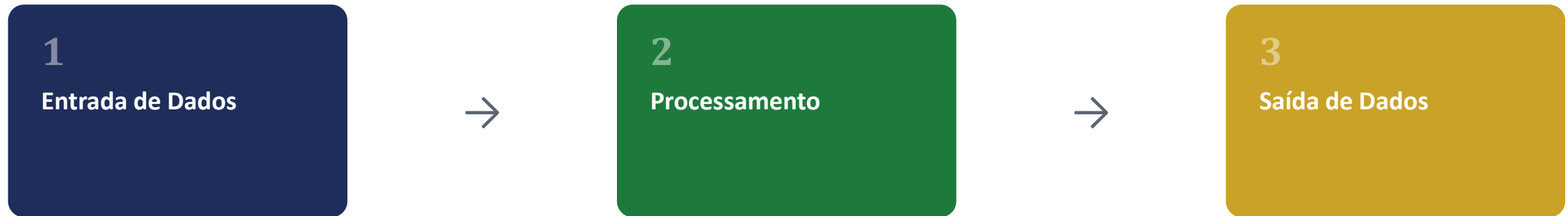
Etapas de um Programa: Entrada, Processamento e Saída

O modelo universal de todo sistema

O modelo Entrada — Processamento — Saída

- 1 Praticamente todo programa de computador segue esse mesmo modelo básico
- 2 Entrada: os dados que o algoritmo recebe para trabalhar
- 3 Processamento: os cálculos, decisões e transformações realizados sobre esses dados
- 4 Saída: o resultado apresentado ao usuário, depois do processamento

O ciclo Entrada — Processamento — Saída



Entrada de dados

- 1 É toda informação que o algoritmo recebe antes de começar a processar
- 2 Pode vir do teclado (comando leia), de um arquivo, de um sensor ou de outro sistema
- 3 No nosso projeto: nome do produto, preço, quantidade e valor pago pelo cliente
- 4 Sem entrada de dados de qualidade, o processamento também será de baixa qualidade

Processamento

- 1 É a etapa em que o algoritmo transforma os dados de entrada em informação útil
- 2 Envolve cálculos (operadores aritméticos), comparações (relacionais) e decisões (lógicos)
- 3 No nosso projeto: calcular o total da compra, verificar estoque, calcular o troco
- 4 É o "cérebro" do algoritmo — onde o raciocínio lógico que estudamos é realmente aplicado

Saída de dados

- 1 É o resultado do processamento, apresentado ao usuário de alguma forma
- 2 Pode ser exibida na tela (comando escreva), impressa, salva em arquivo ou enviada a outro sistema
- 3 No nosso projeto: o comprovante da venda, com o total e o troco calculados
- 4 Uma boa saída de dados é clara, objetiva e fácil de entender pelo usuário final

Relacionando com o diagrama de blocos

- 1 O diagrama de blocos que fizemos hoje também pode ser organizado nesse modelo
- 2 Os primeiros quadros normalmente representam a entrada de dados
- 3 Os quadros do meio representam o processamento (cálculos e decisões)
- 4 Os últimos quadros representam a saída, com o resultado final

O ciclo E-P-S em pseudocódigo

entrada → processamento → saída

Todo algoritmo bem estruturado segue essa ordem básica.

```
// ENTRADA: leia(precoUnitario, quantidade, valorPago)
// PROCESSAMENTO: totalCompra ← precoUnitario * quantidade
//                  troco ← valorPago - totalCompra
// SAÍDA: escreva("Total: ", totalCompra, " Troco: ", troco)
```

RECAPITULANDO

Entrada, Processamento e Saída

- ✓ Entrada: os dados que o algoritmo recebe
- ✓ Processamento: os cálculos, comparações e decisões realizados
- ✓ Saída: o resultado apresentado ao usuário
- ✓ Esse modelo está presente em praticamente todos os módulos do nosso sistema

Instruções da Atividade Avaliativa

- 1 10 questões objetivas, com 5 alternativas cada (a, b, c, d, e)
- 2 Cobrem os três temas da aula: Diagrama de Blocos, Compiladores/Interpretadores e Etapas E-P-S
- 3 Resolvam individualmente, sem consulta, em até 15 minutos
- 4 O gabarito comentado será apresentado logo em seguida

Questões 1 a 5

1

Um diagrama de blocos é usado principalmente para representar:

- a) O código-fonte completo de um programa b) A visão geral das grandes etapas de um sistema
c) Apenas os erros de um algoritmo d) A memória RAM do computador e) Um banco de dados

2

Em relação ao fluxograma, o diagrama de blocos é:

- a) Mais detalhado e complexo b) Mais simples, focado na visão geral c) Exatamente igual, sem diferenças
d) Usado apenas para repetições e) Proibido em algoritmos

3

A única linguagem que o processador executa diretamente é a linguagem:

- a) Portugol b) De máquina (binária) c) Natural (português) d) De marcação (HTML) e) De consulta (SQL)

4

Um tradutor que lê todo o código-fonte de uma vez e gera um arquivo executável é chamado de:

- a) Interpretador b) Compilador c) Depurador d) Sistema operacional e) Navegador

5

Um tradutor que lê e executa o código linha por linha, sem gerar um executável separado, é chamado de:

- a) Compilador b) Montador c) Interpretador d) Roteador e) Servidor

Questões 6 a 10

6

Em um programa compilado, os erros de sintaxe costumam ser detectados:

- a) Somente após a execução completa b) Antes da geração do executável c) Nunca são detectados d) Apenas pelo usuário final e) Somente em linguagens interpretadas

7

O Portugal Studio funciona principalmente como um(a):

- a) Montador (assembler) b) Compilador comercial c) Interpretador educacional d) Sistema operacional e) Banco de dados

8

No modelo Entrada-Processamento-Saída, os cálculos e decisões de um algoritmo ocorrem na etapa de:

- a) Entrada b) Processamento c) Saída d) Compilação e) Depuração

9

O comando "leia(precoUnitario)" representa, no ciclo de um programa, a etapa de:

- a) Processamento b) Saída c) Entrada de dados d) Compilação e) Tradução

10

O comando "escreva(totalCompra)", ao exibir o resultado final ao usuário, representa a etapa de:

- a) Entrada de dados b) Processamento c) Saída de dados d) Montagem e) Interpretação exclusiva

3º

Operadores Relacionais e Lógicos

Comparando valores e tomando decisões

O que são expressões relacionais?

- 1 Comparam dois valores e retornam sempre um resultado lógico: VERDADEIRO ou FALSO
- 2 São a base de toda estrutura de decisão que veremos nas próximas aulas
- 3 Muito usadas para verificar estoque, comparar preços e validar pagamentos
- 4 Em pseudocódigo, usam símbolos parecidos com os da matemática

Operador Maior que (>)

>

Verifica se um valor é maior que outro.

```
totalCompra > 50  
// Se totalCompra = 60 → VERDADEIRO  
// Se totalCompra = 40 → FALSO
```

Operador Menor que (<)

<

Verifica se um valor é menor que outro.

```
estoque < 5  
// Se estoque = 3 → VERDADEIRO  
// Se estoque = 10 → FALSO
```

Operadores $\geq$ e $\leq$

$\geq$ e $\leq$

Verificam "maior ou igual" e "menor ou igual".

```
valorPago  $\geq$  totalCompra    // pagamento suficiente  
estoque  $\leq$  0                // estoque esgotado
```

Operadores = e <>

= e <>

Verificam igualdade e diferença entre dois valores.

```
opcaoMenu = 1          // igual a 1  
precoFornecedor <> precoTabela // diferente
```

O que são operadores lógicos?

- 1 Combinam duas ou mais expressões relacionais em uma única condição
- 2 São três operadores principais: E (AND), OU (OR) e NÃO (NOT)
- 3 Fundamentais para decisões mais completas, como autorizar ou não uma venda
- 4 O resultado de um operador lógico é sempre VERDADEIRO ou FALSO

Operador E (AND)

Só resulta em VERDADEIRO quando as DUAS condições são verdadeiras.

Condição A	Condição B	A E B
Verdadeiro	Verdadeiro	Verdadeiro
Verdadeiro	Falso	Falso
Falso	Verdadeiro	Falso
Falso	Falso	Falso

Operador OU (OR)

Resulta em VERDADEIRO quando PELO MENOS UMA condição é verdadeira.

Condição A	Condição B	A OU B
Verdadeiro	Verdadeiro	Verdadeiro
Verdadeiro	Falso	Verdadeiro
Falso	Verdadeiro	Verdadeiro
Falso	Falso	Falso

Operador NÃO (NOT)

Inverte o valor lógico de uma condição.

Condição A	NÃO A
Verdadeiro	Falso
Falso	Verdadeiro

Combinando E, OU e NÃO

- 1 É possível usar mais de um operador lógico na mesma expressão
- 2 Parênteses ajudam a deixar claro o que deve ser avaliado primeiro
- 3 Exemplo: NÃO (estoque = 0) → verdadeiro quando HÁ estoque disponível
- 4 Quanto mais condições, mais importante é organizar bem a expressão

Exemplo real: estoque E pagamento

`(estoque >= quantidade) E (valorPago >= totalCompra)`

A venda só é autorizada se as duas condições forem verdadeiras ao mesmo tempo.

```
SE (estoque >= quantidade) E (valorPago >= totalCompra) ENTÃO  
    escreva("Venda autorizada")  
SENÃO  
    escreva("Venda não autorizada")
```

4º

Atividade Prática em Duplas

Elaborando expressões para 3 situações de venda

Atividade Prática em Duplas — Instruções

- 1 Em duplas, elaborem em papel as expressões (aritméticas, relacionais e lógicas) das 3 situações a seguir
- 2 Usem os exemplos desta aula como referência
- 3 Cada expressão deve estar em pseudocódigo, pronta para ser digitada depois no Portugol Studio
- 4 Tempo: 20 minutos — corrigiremos juntos(as) no quadro em seguida

1

Situação 1 — Cálculo de compra

Um cliente compra 4kg de farinha (R\$ 6,50/kg) e 2 potes de açaí (R\$ 12,00/pote). Escrevam a expressão que calcula o total da compra.

2

Situação 2 — Verificação de troco

O cliente paga R\$ 50,00 pela compra da situação anterior. Escrevam a expressão que verifica se o valor pago é suficiente e calcula o troco.

3

Situação 3 — Venda condicionada

Há 6 unidades de açaí em estoque e o cliente quer comprar 2. Escrevam a expressão lógica que verifica se HÁ estoque suficiente E se o cliente pagou o valor correto.

Competência Técnica desta Aula

Elaborar algoritmos lógicos de forma estruturada, em conformidade com padrões de nomenclatura e convenções de linguagem, a fim de solucionar problemas computacionais, promovendo uma base técnica em lógica computacional e suas aplicações.

Habilidades e Objetos de Conhecimento

HABILIDADE

Manipular com atenção os tipos de dados e constantes em algoritmos, aplicando conversões na garantia da integridade e eficiência do processamento.

OBJETO

Tipos de Dados: Lógico, Numérico: Inteiro, Real, Caractere, Conversão entre tipos, Constantes.

HABILIDADE

Declarar variáveis em programas, diferenciando local e global, garantindo a correta organização e funcionalidade do código.

OBJETO

Variáveis: Identificador, Variável local e Variável global.

05

Tipos de Dados e Constantes

1º e 2º Tempos

Os 4 tipos de dados primitivos

Numéricos

- Inteiro — números sem casas decimais
- Real — números com casas decimais

Não numéricos

- Lógico — verdadeiro ou falso
- Caractere — texto, letras e símbolos

Tipo Lógico

lógico

Armazena apenas dois valores possíveis: VERDADEIRO ou FALSO. Também chamado de booleano.

```
disponivel ← VERDADEIRO  
cooperado ← FALSO
```

Tipo Numérico Inteiro

inteiro

Armazena números sem casas decimais — usado para contagens e quantidades.

```
quantidadeEstoque ← 25  
numeroVendas ← 8
```

Tipo Numérico Real

real

Armazena números com casas decimais — usado para valores que exigem precisão.

```
precoPeixe ← 18.50  
pesoAcai ← 2.75
```

Inteiro x Real — não confunda!

Inteiro

- Sem casas decimais
- Ex.: quantidade = 3
- Uso: contagens, unidades

Real

- Com casas decimais
- Ex.: preço = 3.50
- Uso: valores monetários, medidas

Tipo Caractere (Texto)

caractere

Armazena letras, palavras ou frases — sempre entre aspas em pseudocódigo.

```
nomeProduto ← "Peixe Tambaqui"  
nomeCliente ← "Maria"
```

Resumo: os 4 tipos de dados

Tipo	Armazena	Exemplo
Lógico	Verdadeiro ou Falso	disponivel ← VERDADEIRO
Inteiro	Números sem casas decimais	quantidade ← 25
Real	Números com casas decimais	preco ← 18.50
Caractere	Texto, letras, símbolos	nome ← "Peixe"

O que é uma constante?

- 1 É um valor que NÃO muda durante toda a execução do algoritmo
- 2 Diferente da variável, que pode ser alterada a qualquer momento
- 3 Usada para valores fixos e conhecidos previamente, como taxas e limites
- 4 Ajuda a evitar erros: o valor não pode ser alterado por engano em outra parte do código

Constante x Variável

Constante

- Valor fixo, não muda
- Definida uma única vez
- Ex.: taxaComissao $\leftarrow$ 0.05

Variável

- Valor pode mudar
- Atualizada quantas vezes for preciso
- Ex.: totalVendasDia $\leftarrow$ 150.00

RECAPITULANDO

Tipos de Dados e Constantes

- ✓ Os 4 tipos primitivos: lógico, inteiro, real e caractere
- ✓ Conversão de tipos permite transformar um valor de um tipo em outro
- ✓ Constante é um valor fixo, que não muda durante a execução
- ✓ Escolher o tipo certo é a base para variáveis confiáveis e cálculos corretos

06

Variáveis

3º e 4º Tempos — Identificador, Escopo Local e Global

O que é uma variável? (recapitulando)

- 1 É um espaço na memória do algoritmo que guarda um valor durante a execução
- 2 Toda variável tem um nome (identificador), um tipo de dado e um valor
- 3 O valor de uma variável pode ser alterado quantas vezes for necessário
- 4 Já usamos variáveis desde a Aula 01 — agora vamos entender suas regras a fundo

O que é um identificador?

- 1 É o nome que damos a uma variável (ou constante) dentro do algoritmo
- 2 Deve representar claramente o que aquela variável armazena
- 3 Segue regras específicas de formação, que veremos a seguir
- 4 Um bom identificador facilita a leitura do algoritmo por qualquer pessoa

Regras de nomenclatura de identificadores

- 1 Não pode começar com número (ex.: 1produto é inválido)
- 2 Não pode conter espaços em branco (use camelCase ou underline)
- 3 Não pode conter caracteres especiais como @, #, \$, -
- 4 Não pode ser igual a uma palavra reservada da linguagem (ex.: SE, ENQUANTO)

Identificadores válidos x inválidos

Válidos

- precoProduto
- qtdEstoque
- nomeCliente_1

Inválidos

- 1produto (começa com número)
- preço-produto (caractere especial)
- nome cliente (espaço em branco)

Convenção camelCase

camelCase

Primeira palavra em minúscula, demais palavras iniciando em maiúscula — sem espaços.

```
nomeProduto  
precoUnitario  
quantidadeEstoqueAtual
```

O que é o escopo de uma variável?

- 1 Escopo é a "área" do algoritmo onde uma variável pode ser acessada e utilizada
- 2 Nem toda variável está disponível em todas as partes do algoritmo
- 3 Existem dois tipos principais de escopo: local e global
- 4 Entender escopo evita erros difíceis de identificar em algoritmos maiores

Variável Local

escopo local

Declarada dentro de um módulo/procedimento específico — só existe e só pode ser usada ali dentro.

```
procedimento calcularSubtotal()  
    subtotalItem ← quantidade * precoUnitario  
    // subtotalItem só existe aqui dentro  
fim procedimento
```

Variável Global

escopo global

Declarada fora de qualquer módulo — pode ser acessada e alterada em qualquer parte do algoritmo.

```
GLOBAL totalVendasDia ← 0.00

procedimento registrarVenda()
    totalVendasDia ← totalVendasDia + subtotalItem
fim procedimento
```

Variável Local x Variável Global

Local

- Existe só dentro do módulo
- Não interfere em outras partes
- Ideal para cálculos temporários

Global

- Existe em todo o algoritmo
- Pode ser alterada por qualquer módulo
- Ideal para totais e acumuladores

Quando usar variável local?

- 1 Quando o valor é temporário e usado apenas dentro de um cálculo específico
- 2 Quando você quer evitar que outras partes do algoritmo alterem esse valor por engano
- 3 A maioria das variáveis de um algoritmo bem estruturado deveria ser local
- 4 Isso torna o código mais organizado e mais fácil de depurar

Quando usar variável global?

- 1 Quando o valor precisa ser compartilhado e atualizado por diferentes módulos
- 2 Quando representa um estado geral do sistema (ex.: total do dia, usuário logado)
- 3 Use com moderação — o uso excessivo de variáveis globais é considerado uma má prática
- 4 Sempre pergunte: "esse valor realmente precisa estar disponível em todo lugar?"

Riscos do uso excessivo de variáveis globais

- 1 Qualquer módulo pode alterar o valor, dificultando saber "quem mudou o quê"
- 2 Aumenta a chance de um módulo sobrescrever, sem querer, um valor usado por outro
- 3 Torna o algoritmo mais difícil de testar e de dar manutenção
- 4 Regra geral: prefira variáveis locais sempre que possível

Erros comuns com variáveis

- 1 Usar o mesmo nome para uma variável local e uma global, causando confusão
- 2 Declarar como global uma variável que deveria ser local
- 3 Esquecer de inicializar uma variável antes de usá-la em um cálculo
- 4 Alterar o valor de uma variável em um módulo sem perceber o efeito em outro

Declarando nomeProduto

```
nomeProduto : caractere (local)
```

Usada apenas dentro do módulo de cadastro.

```
nomeProduto ← "Farinha de Mandioca"
```

Declarando precoUnitario

```
precoUnitario : real (local)
```

Usada apenas dentro do módulo de cadastro.

```
precoUnitario ← 9.50
```

Declarando quantidadeEstoque

```
quantidadeEstoque : inteiro (local)
```

Usada apenas dentro do módulo de cadastro.

```
quantidadeEstoque ← 60
```

Declarando disponivel

```
disponivel : lógico (local)
```

Indica se o produto está disponível para venda.

```
disponivel ← VERDADEIRO
```

Juntando tudo: módulo de cadastro de produto

pseudocódigo completo

O primeiro módulo do nosso sistema, unindo tipos de dados e variáveis locais.

```
procedimento cadastrarProduto()  
    nomeProduto ← "Farinha de Mandioca"  
    precoUnitario ← 9.50  
    quantidadeEstoque ← 60  
    disponivel ← VERDADEIRO  
    escreva("Produto cadastrado: ", nomeProduto)  
fim procedimento
```

Testando o algoritmo (teste de mesa)

- 1 Executem mentalmente o pseudocódigo da tela anterior, linha por linha
- 2 Anotem o valor de cada variável após cada linha executada
- 3 Confirmem que a mensagem final exibida está correta
- 4 Essa técnica (teste de mesa) ajuda a encontrar erros antes mesmo de rodar no computador

RECAPITULANDO

Variáveis

- ✓ Identificador é o nome da variável — deve seguir regras claras de formação
- ✓ Variável local existe apenas dentro do módulo onde foi declarada
- ✓ Variável global existe e pode ser alterada em todo o algoritmo
- ✓ Prefira variáveis locais; use globais apenas quando realmente necessário

Como registrar sua expressão

- 1 Declarem primeiro as variáveis usadas (com seus valores)
- 2 Escrevam a expressão completa, com os parênteses necessários
- 3 Confiram a ordem de precedência antes de considerar a resposta pronta
- 4 Estejam prontos(as) para explicar seu raciocínio na correção coletiva

Instruções da Atividade Avaliativa

- 1 10 questões objetivas, com 5 alternativas cada (a, b, c, d, e)
- 2 Cobrem o conteúdo da aula: atribuição, operadores aritméticos, relacionais e lógicos
- 3 Resolvam individualmente, sem consulta, em até 15 minutos
- 4 O gabarito comentado será apresentado logo em seguida

Questões 1 a 5

1

Em pseudocódigo, o operador de atribuição é representado por:

a) = b) $\leftarrow$ (ou $:=$) c) == d) => e) ;

2

Qual o valor de total, na expressão: $\text{total} \leftarrow 12 + 3 * 2$?

a) 30 b) 18 c) 17 d) 15 e) 21

3

Um cliente compra 3kg de peixe a R\$ 15,00/kg. Qual expressão calcula o total?

a) $\text{total} \leftarrow 3 + 15$ b) $\text{total} \leftarrow 3 * 15$ c) $\text{total} \leftarrow 15 / 3$ d) $\text{total} \leftarrow 3 - 15$ e) $\text{total} \leftarrow 15 \% 3$

4

Qual operador retorna o resto de uma divisão?

a) / b) * c) % d) + e) -

5

Qual operador relacional representa "maior ou igual a"?

a) > b) >= c) < d) <= e) <>

Questões 6 a 10

6

Considere estoque = 8 e pedido = 10. Qual expressão resulta em VERDADEIRO?

- a) estoque >= pedido b) estoque = pedido c) pedido > estoque d) estoque > pedido e) estoque <> estoque

7

O operador lógico E (AND) retorna verdadeiro quando:

- a) Pelo menos uma condição é verdadeira b) Todas as condições são verdadeiras c) Nenhuma condição é avaliada d) A primeira condição é falsa e) Sempre retorna verdadeiro

8

O operador lógico OU (OR) retorna falso quando:

- a) Todas as condições são falsas b) Pelo menos uma condição é verdadeira c) A primeira condição é verdadeira d) Existe uma condição nula e) Nunca retorna falso

9

Qual o resultado de: (10 > 5) E (3 > 8) ?

- a) Verdadeiro b) Falso c) 10 d) 3 e) Erro de sintaxe

10

Uma venda só é autorizada se HÁ estoque suficiente E o pagamento é suficiente. Isso é um exemplo de uso de:

- a) Operador aritmético isolado b) Operador de atribuição c) Operadores lógicos combinando condições relacionais d) Conversão de tipos e) Comentário de código

Encerramento da Aula 04

- Hoje: atribuição, operadores aritméticos, relacionais e lógicos
- Guardem as expressões construídas hoje — elas serão reaproveitadas no módulo de vendas
- Obrigada pela participação de todos(as)!